



University
of Stavanger

LUKAS FERNANDEZ BERNTSEN AND MARKUS SOLGÅRD
DEPARTMENT OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

Design, production, testing, and integration of a user interface for a suborbital rocket

Bachelor's Thesis - Computer Science - May 2026



UIS AEROSPACE

Declaration on the Use of AI Tools for Assignment Submission

Name	Lukas Fernandez Berntsen og Markus Solgård
Course Code or Name	DATBAC
Semester:	Vår 2026

At UiS, it is permitted to use AI-based tools properly unless otherwise stated in the course description or assignment.

In general, proper use of AI tools means that these tools are used as an aid. The response is prepared by the students themselves, and sources are cited in accordance with the regulations. Bachelor's and master's theses, as well as other submission assignments, are written and independent tasks. You are responsible for the final text submitted for assessment.

For more information on the proper use of AI tools: [Examination | University of Stavanger](#)

Fill out the form to indicate whether you have used AI tools in your assignment.

The purpose of describing this use is to show the reader how you have worked in your assignment. By doing this, you follow the requirements of transparency and integrity in an academic assignment, and it may be easier for the assessor to evaluate your own contribution. By being clear about how you proceeded, the assessor will get an explanation of how you have used language models in your assignment.

Language editing: Have you used generative artificial intelligence for proofreading/correction of parts or the entire text?

☒ Yes

☐ No

If yes, specify which tools you have used and briefly describe how you have used them in your assignment:

We have used the language model ChatGPT to correct spelling and grammatical errors, and to get suggestions for simplifying and clarifying some paragraphs. We have used these suggestions as support, but the final wording has been reviewed, edited, and controlled by us.

Brainstorming: Have you used generative artificial intelligence for ideas and suggestions for the structure of your assignment?

☒ Yes

☐ No

If yes, specify which tools you have used and briefly describe how you have used them in your assignment:

We have used ChatGPT for brainstorming ideas and suggestions for how to structure specific paragraphs in the thesis. The overall structure of the assignment was based on previous bachelor theses and discussions with our advisor.

Images and figures: Have you generated one or more of the images/figures in the assignment using generative artificial intelligence?

☐ Yes

☒ No

☐ Not relevant

If yes, specify which tools you have used and briefly describe how you have used them in your assignment:

Codes and algorithms: Have you generated parts of the code/algorithms that 1) appear directly in the thesis/assignment or 2) have been used to produce results such as figures, tables, or numerical values, using generative artificial intelligence?

- ☒ Yes
☐ No
☐ Not relevant

If yes, specify which tools you have used and briefly describe how you have used them in your assignment:

The language models Claude and ChatGPT were used as support tools for debugging. The autofill/autocomplete tool in VS Code was also used during development. However, all final code was written, reviewed, and controlled by us. No tables, figures, or numerical values in the thesis were generated using AI.

Other AI aids og tools: Have you used other types of AI aids or tools in the creation of this assignment?

- ☐ Yes
☒ No

If yes, specify which tools you have used and briefly describe how you have used them in your assignment:

I confirm that I have read the information on the use of AI tools in connection with assignment writing and exams.
[Examination | University of Stavanger](#)

How to fill out the form if you have used generative AI/language models in your assignment:

Here are some examples of how a description might look:

1. I have used the language model Copilot in this assignment for inspiration on the structure of my text. I used Copilot to suggest an outline, but I have only taken ideas from this outline, which I have reviewed and processed myself.
2. I have used the language model Copilot in this assignment for inspiration on the argumentation in my text. I have received input to develop arguments in the discussion section. I have not used Copilot to write entire paragraphs or sentences in the assignment. I have only taken ideas that I have reviewed and processed myself based on the syllabus material and other scientific sources I have searched for without using AI.
3. I have used Copilot to correct spelling and other grammatical errors and to ask for suggestions to simplify and clarify some short paragraphs. I have taken inspiration from these suggestions but written my own sentences instead of copying and pasting from the AI suggestions.

I, **Lukas Fernandez Berntsen and Markus Solgård**, declare that this thesis titled, “Design, production, testing, and integration of a user interface for a suborbital rocket” and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a bachelor’s degree at the University of Stavanger.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.

Abstract

UiS Aerospace is a student-led organization pursuing the long-term goal of reaching space, with the Phoenix 2 rocket representing an important step toward that objective. As the organization's rockets, propulsion systems, and electrical subsystems become more advanced, the demands placed on supporting software also increase. This thesis presents the design, implementation, testing, and integration of a ground control interface intended to improve reliability, flexibility, and long-term maintainability in testing and launch operations.

The project resulted in a distributed system with separate interfaces for operators and external viewers. The operator interface supports live telemetry monitoring, replay of previously recorded sessions, and command handling, while a public-facing application presents live mission data to sponsors and other observers. The system was implemented using React, .NET, SignalR, Redux, Docker, and InfluxDB, with telemetry stored both locally and on external servers.

The completed system provides a more modular and reusable foundation for ground control operations and post-flight analysis, and is intended to support the continued development of future UiS Aerospace missions.

Abbreviations & Expressions

API Application Programming Interface

CAN Controller Area Network

CLI Command Line Interface

COPV Composite Overwrapped Pressure Vessel

CSS Cascading Style Sheets

DB Database

DOM Document Object Model

GUI Graphical User Interface

HTML Hypertext Markup Language

HTTP Hypertext Transfer Protocol

JSX JavaScript XML

npm Node Package Manager

N₂O Nitrous Oxide

OCI Open Container Initiative

REST Representational State Transfer

SDK Software Development Kit

SQL Structured Query Language

TCP Transmission Control Protocol

UiS University of Stavanger

URL Uniform Resource Locator

Acknowledgements

We would first like to thank the UiS Aerospace team for their collaboration and for giving us the valuable opportunity to work together on this project. We would also like to thank the University of Stavanger for providing us with the education and guidance necessary to undertake this work. Finally, we would like to extend our sincere gratitude to our advisor, Erlend Tøssebro, for his valuable input and time.

Repository

The code for this project is available at:

<https://github.com/LBerntsen/bachelor-aerospace-gui>

Contents

Abstract	ii
Abbreviations & Expressions	iii
Acknowledgements	v
Repository	vi
1 Introduction	1
1.1 UiS Aerospace	1
1.2 Bachelor's Theses in Avionics	6
1.3 Background	7
1.4 Identifying a Problem	7
1.5 Proposed Solution	8
1.6 Objectives	9
1.7 Outline	10
2 Background	11
2.1 Frontend Technologies	11

2.1.1	React and Component-Based Architecture	11
2.1.2	State Management	12
2.1.3	Tailwind CSS for Responsive Design	13
2.2	Backend Technologies	13
2.2.1	Real-time Communication via SignalR	13
2.2.2	The .NET Framework	14
2.2.3	Containerization of the application	15
2.3	Data Management	16
2.3.1	Telemetry as Time-Series Data	16
2.3.2	Persistent Storage: InfluxDB vs. Relational Databases . . .	16
2.4	System Design Considerations	17
2.4.1	Decoupling and Modularity	17
2.4.2	Challenges in Real-Time Systems	18
2.5	Hosting and Deployment	18
2.6	2025 Thesis	19
3	System Design and Implementation	21
3.1	System Overview	21
3.2	Architecture	24
3.3	Backend Implementation	25
3.4	Frontend Implementation	28
3.5	Telemetry Storage and Replay	32
3.6	Public Website	32
3.7	Security	33

3.8	Command Handling	34
3.9	Deployment and Development Setup	35
3.10	Chapter Summary	35
4	Results	36
4.1	Experimental Setup and Data Set	36
4.2	Integration testing with the telemetry group	38
4.3	Testing of the sponsor functionality	38
4.4	Dashboards	39
4.4.1	Main Dashboard	39
4.4.2	Propulsion	42
4.4.3	Avionics	42
4.4.4	Recovery	43
4.4.5	Start, Stop, and Replay Module	44
5	Discussion	46
5.1	Comparing with previous project	46
5.2	Working with the other groups and launch postponement	47
5.3	Result Evaluation	48
5.4	Future Improvements	49
6	Conclusion	50
A	Instructions to Compile and Run the System	55
A.1	System Requirements	55
A.2	Project Structure	56

A.3	Configuration	56
A.4	Running the Operator Version	57
A.5	Running the Sponsor Version	57
A.6	Running the Backend and Frontend Separately	57
A.6.1	Backend	57
A.6.2	Frontend	58
A.7	Replay Functionality	58
A.8	Troubleshooting	58
B	Adding New Charts and Dashboard Elements	59
B.1	Overview	59
B.2	Creating a New Dashboard Page	60
B.3	Using Existing Components	61
B.4	Grouping Components with Cards	62
B.5	Adding a New Telemetry Value	63
B.6	Recommended Development Process	63
B.7	Troubleshooting	64
C	Excel Overview Sheet	66

Chapter 1

Introduction

Note that the following Section 1.1 is written as a collaborative effort by multiple bachelor groups within UiS Aerospace, and will be submitted in multiple different theses.

1.1 UiS Aerospace

UiS Aerospace is a student engineering organization at UiS. The team designs, manufactures, and launches research rockets, and offers students from different engineering disciplines the opportunity to collaborate on advanced aerospace projects. The organization consists of approximately 100 members divided into specialized groups covering propulsion, avionics, mechanical and admin. Most projects are carried out across groups, creating a multidisciplinary environment.



Figure 1.1: Members of the UiS Aerospace team at the launch site on the island of Utsira, prior to the launch of Phoenix 1 on 21 June 2025.

Prior to the Phoenix program, UiS Aerospace developed two high-power solid-propellant rockets through the Borealis program. Borealis 1 and Borealis 2 were launched in 2023 and 2024 respectively. The program established the team's core infrastructure and developed several key technologies, including avionics, recovery systems, and the rib structure used to integrate subsystems across airframe sections.

The Phoenix program is UiS Aerospace's liquid bi-propellant rocket program, with the long-term goal of reaching the Kármán line at 100 km. Phoenix 1 is the first rocket in this program, built to validate the propulsion system, avionics, and launch procedures required for that mission.

Phoenix 1 was an eight meter tall rocket built around the Berserkr 310 engine, a liquid bi-propellant engine developed in-house. The engine burns ethanol as fuel and N_2O as oxidiser, with a chamber pressure of 60 bar and a thrust of 16 kN. Propellant is stored in a common-bulkhead COPV, an aluminum liner overwrapped

in carbon fibre to withstand the high feed pressure. Phoenix 1 also carried student-built $10 \times 10 \times 10$ cm satellites from local secondary and high schools as part of the UiS Aerospace Rideshare program, introducing students to engineering and space technology.

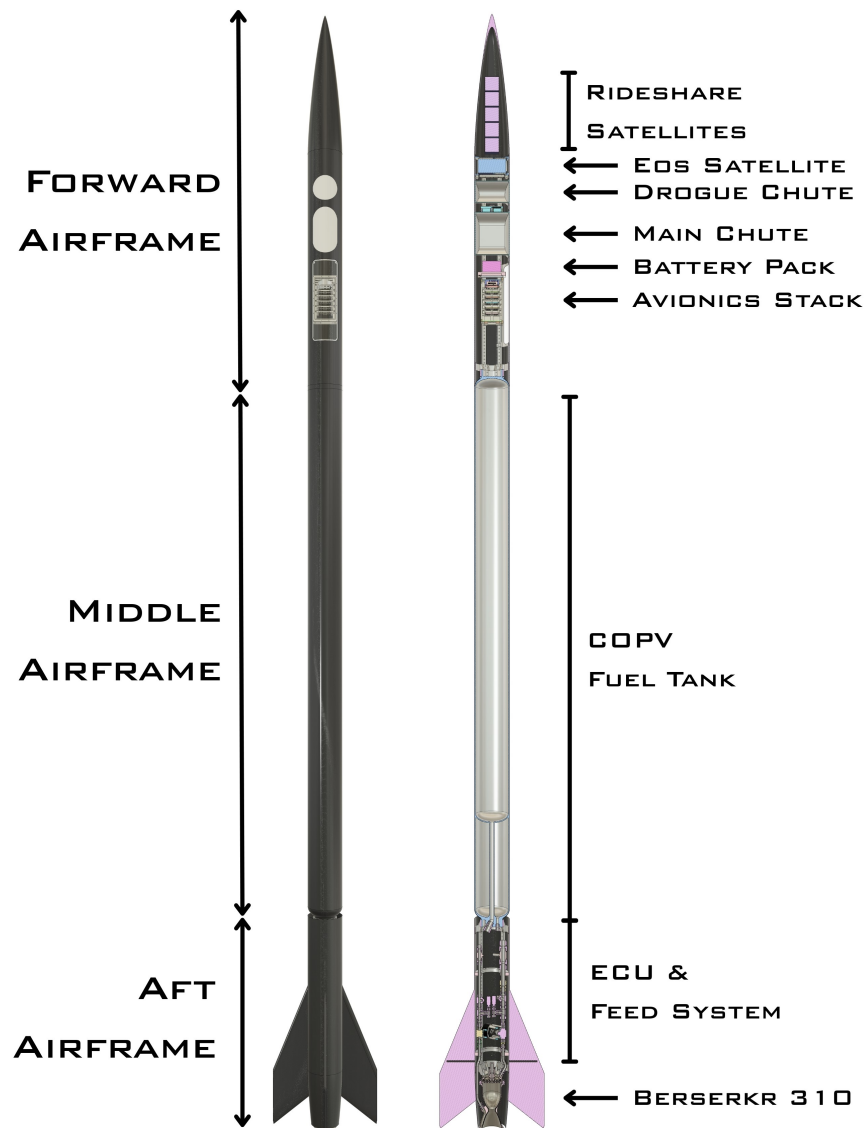


Figure 1.2: Cross section of Phoenix 1 showing the three part airframe structure and key subsystems. Figure made by UiS Aerospace.

On 21 June 2025, Phoenix 1 was launched from the island of Utsira on the Norwegian west coast. The launch was the first liquid bi-propellant student rocket launch from Norwegian soil.



Figure 1.3: Phoenix 1 launch from Utsira.

The aft airframe and propellant tank were recovered after the flight. The forward airframe, containing the avionics, recovery, and rideshare sections, most likely separated on water impact and was not recovered. A recap of the launch and project can be seen in the official Phoenix 1 aftermovie [1].



Post-flight inspection of the Berserkr 310 engine revealed a blow-through of the chamber wall, suspected to be caused by a failed cooling channel, possibly combined with a fuel-rich combustion mixture increasing internal wall temperatures beyond the design limit.

The Phoenix program has produced the test infrastructure used in this thesis. UiS Aerospace constructed a 20-foot shipping container as a static firing facility, fitted with a thrust plate, aft-airframe mount, and propellant feed systems. This container only has small changes for Phoenix 2 and serves as the ground test facility for the electrical system described in this thesis.

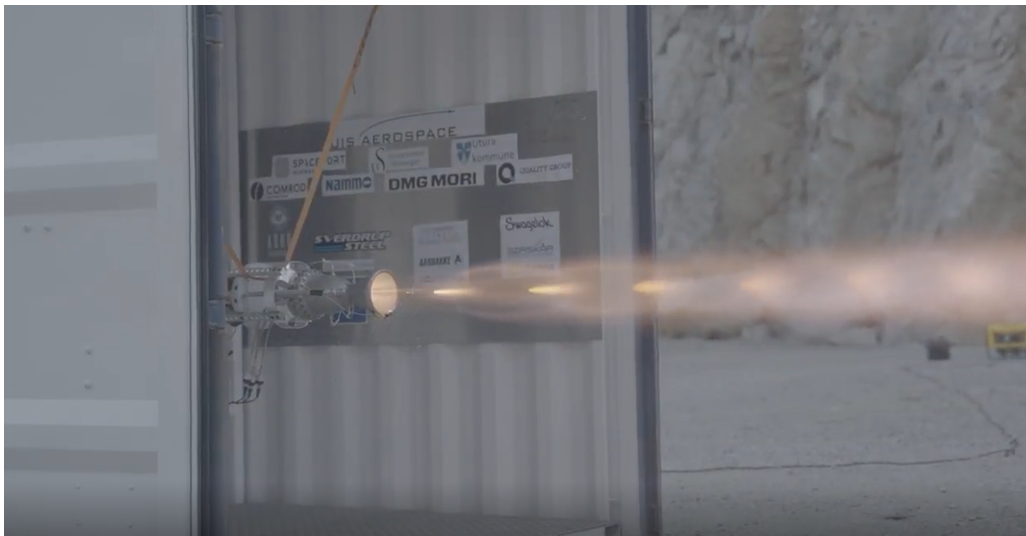


Figure 1.5: The test container in use during hotfire testing for Phoenix 1.

1.2 Bachelor's Theses in Avionics

The Phoenix 2 mission is a large interdisciplinary effort involving multiple student groups across different engineering disciplines. Within the avionics team, four bachelor groups are contributing to the 2025/2026 academic year, each responsible for developing a distinct subsystem critical to the mission. The avionics team works on the rocket's electrical systems, spanning areas such as communication, sensing, propulsion support, and ground operations.

Telemetry

Responsible for the radio link between the rocket and the ground team, handling both the downlink of sensor data and the uplink of commands sent to the rocket.

Sensors and Camera

Collects temperature data in the nosecone and strain measurements in the ribs during flight, providing data for post-flight structural analysis. The group records onboard imagery throughout the mission.

Turbopumps

Develops and integrates the turbopump system responsible for feeding propellant to the engine at the required pressure and flow rate.

Ground Control Interface

Receives telemetry from the telemetry system, presents it to operators in real time, and provides the interface for sending commands to the rocket.

1.3 Background

UiS Aerospace is preparing for the launch of its fourth rocket, with the goal of reaching the Kármán line, representing another step in the organization's technical development. As the rockets and their subsystems become more advanced, the demands placed on the supporting software also increase. New missions introduce additional sensors, updated telemetry formats, and changing operational requirements, which place greater demands on the systems used for monitoring and control.

Ground control software plays a central role in this context. It enables operators to monitor subsystem status, interpret telemetry in real time, and issue commands when necessary. Because such software is used during testing and mission-critical phases, it must not only provide low-latency access to data, but also remain reliable, understandable, and easy to extend as mission requirements evolve.

For student-led projects such as UiS Aerospace, maintainability is especially important. Software developed by one group must often be continued by later groups with limited prior knowledge of the implementation. This creates a need for solutions that are not tightly bound to one mission profile, one telemetry structure, or one specific set of sensors, but instead can be adapted over time without major redesign.

1.4 Identifying a Problem

This section is partly based on the findings from a previous bachelor thesis in UiS Aerospace, which developed a web-based ground control system for displaying telemetry and communicating with the rocket [2]. That work made a functional system, but it also had several limitations.

Telemetry data was stored locally, which made it less accessible after tests and missions and created dependence on a single machine. The thesis pointed out the need for server-based storage, but this was not completed during the project period. In addition, the system was closely connected to the telemetry and

communication setup used at the time. As a result, changes in telemetry formats or subsystem requirements could require changes in several parts of the application.

As UiS Aerospace develops new rockets and new subsystems, future missions are likely to require support for new sensors, updated signal definitions, and different operational needs. A system that is difficult to extend, maintain, and reuse will therefore limit further development. This thesis addresses that problem by designing a ground control application that is more flexible and better suited for future missions.

1.5 Proposed Solution

To address the limitations identified in earlier work, this project proposes a new ground station system that is easier to maintain, easier to adapt to new mission requirements, and better suited for continued use in UiS Aerospace. The goal is not only to support current launch operations, but also to provide a foundation that can be reused and improved in future missions.

A central part of the proposed solution is persistent storage of telemetry data on UiS Unix servers. Instead of depending on local storage on a single machine, mission data will be stored centrally and remain available after tests and launches. This makes it easier to review data afterwards, compare results across missions, and reduce the risk of losing access to important information.

The proposed solution also includes a clearer separation between different parts of the system. This is intended to make the application less dependent on one specific telemetry setup and easier to adapt when sensors, signal definitions, or subsystem requirements change. In practice, this means the system should require fewer changes when new mission needs arise.

In addition to supporting internal operations, the solution will make the latest launch data available on a website hosted on the UiS Unix servers. This will allow sponsors and other observers to follow the launch and view relevant telemetry during missions. In this way, the system is intended not only as an operational tool for the team, but also as a way to share mission progress with external stakeholders.

The system will also support command handling for mission operations. This is necessary for practical use during testing and launch, while still requiring appropriate access control. The overall aim is to provide a ground station system that is more useful during missions, more practical to build on afterwards, and better suited to the long-term development of UiS Aerospace.

1.6 Objectives

The primary goal of this thesis is to improve the Ground Control Station by supporting both live operations and post-flight analysis. The specific objectives are:

- Ensure reliable real-time handling of telemetry data during missions.
- Store telemetry data persistently on UiS servers.
- Enable replay and analysis of recorded flight data.
- Provide visualization tools that are reusable and adaptable to different mission needs.
- Provide launch data on a website, so that sponsors and other observers can follow the mission.

1.7 Outline

The remainder of this thesis is structured as follows:

- **Chapter 2: Background** presents the theoretical foundation.
- **Chapter 3: Approach** describes the system design and implementation.
- **Chapter 4: Evaluation** presents testing and results.
- **Chapter 5: Discussion** reflects on the results and limitations.
- **Chapter 6: Conclusion** summarizes the work.

Chapter 2

Background

This thesis assumes that the reader has a fundamental understanding of computer science and related technologies. This includes basic knowledge of programming languages such as Python, HTML, CSS, SQL, and C#, and knowledge of frameworks such as, Bootstrap, Vue, and .NET. This chapter builds upon that foundation by presenting the concepts and technologies relevant to the system developed in this project.

2.1 Frontend Technologies

The frontend must display live telemetry data in a clear and extensible interface, placing high demands on the chosen technologies. This section outlines the frameworks and tools selected to meet those requirements.

2.1.1 React and Component-Based Architecture

When selecting a frontend framework, Vue was evaluated as an alternative to React. Both frameworks share a component-based architecture and use a virtual DOM for efficient rendering. Vue is generally considered more beginner-friendly than React, making it well suited for rapid prototyping and smaller teams [3]. However, React has a significantly larger ecosystem, offering more third-party

libraries, tooling, and long-term corporate support from Meta, making it better suited for complex and scalable applications [4]. Given the complexity and extensibility requirements of this project, including real-time dashboards and multiple reusable components, React was chosen over Vue, also due to prior experience with React from earlier projects.

In a ground control system, the larger ecosystem, and more third-party libraries makes it easier to build modular dashboards. Different elements, such as charts, gauges, and status indicators, can be developed and tested independently. Because of the virtual DOM, React allows efficient updates when data changes. This is important when handling telemetry data that updates frequently.

2.1.2 State Management

State management describes how an application stores, updates, and shares data used by the user interface [5]. In this project, it is especially important because several parts of the frontend depend on the same telemetry data and must remain synchronized.

Two alternatives were considered: prop drilling and Redux. Prop drilling refers to passing data through multiple levels of the component tree via props [6]. In smaller applications, this can be a practical solution because it is simple and does not require additional libraries. However, as the number of components and shared data dependencies increases, this approach can make the structure harder to follow and maintain.

Redux, by contrast, stores shared state in a centralized store that can be accessed by components regardless of their position in the hierarchy. This makes it better suited for applications where data changes frequently and must be used in several places at once [7].

Because this system processes continuously updated telemetry data that is consumed by many components, Redux was chosen over prop drilling. A centralized state model improves consistency, reduces unnecessary coupling between components, and provides a more scalable foundation for further development [8]. In practice, incoming telemetry is stored in the Redux store,

and connected components are updated automatically when new values arrive. This simplifies data flow in the frontend and makes it easier to extend the system with new views or telemetry sources.

2.1.3 Tailwind CSS for Responsive Design

When selecting a CSS framework the choice was between Bootstrap and Tailwind, as Bootstrap was taught at the University of Stavanger it was the natural choice to make. According to [9] it provides ready-made components that can be used without writing custom CSS, making it beginner-friendly and fast to get started with, but at the cost of customization. Tailwind on the other hand is much more customizable as it comes with a large set of utility classes used for styling that can be mixed and matched. The amount of utility classes gives Tailwind a steeper learning curve but more possibilities, and the unused classes get removed keeping the final CSS bundle lightweight. Despite prior familiarity with Bootstrap, Tailwind CSS was selected as it better aligned with the project requirements. This choice was driven by the need for a clear and readable interface for the ground control system while matching the UiS Aerospace branding.

2.2 Backend Technologies

The backend of this system is responsible for receiving telemetry data, processing it, and delivering it to the frontend with minimal latency. This section covers the technologies chosen to fulfill these demands.

2.2.1 Real-time Communication via SignalR

Three approaches were evaluated for delivering telemetry data to the frontend in real time: REST polling, raw WebSockets, and SignalR.

REST polling is the simplest alternative, where the client periodically sends HTTP requests to check for new data. However, polling introduces latency equal to the polling interval and can cause significant server load when many clients are

polling simultaneously, making it poorly suited to genuine real-time applications [10].

Raw WebSockets were also considered, as they provide minimal latency once a connection is established. The connection enable the client and server to push data directly between each other, making WebSockets ideal for high-frequency, low-latency applications. The downside is that WebSocket is a low-level protocol where developers must manage state, connection, reconnection and fallback mechanisms themselves [11].

SignalR was ultimately chosen because it combines the performance of WebSockets with significantly reduced implementation complexity. SignalR automatically negotiates the best available transport protocol, starting with WebSockets and falling back to Server-Sent Events or Long Polling if necessary, abstracting away the complexities that developers must handle using WebSockets. SignalR also uses a high-level pipeline called the SignalR Hubs API, which allows clients and servers to communicate directly [12]. Since the backend is built on .NET, SignalR integrates naturally into the existing tech stack. SignalR also provides a dedicated package for React on the client side [11].

In this project, SignalR is used to stream telemetry data from the backend to the frontend in real time. Instead of relying on repeated client requests it utilizes WebSockets to maintain a persistent connection between the client and server, allowing data to be sent as soon as it becomes available.

This is important for telemetry systems, where low latency is required. Using WebSockets, the backend can push data directly to the frontend without delay.

2.2.2 The .NET Framework

Another choice which had to be made was between Flask or .NET for the backend, which also meant choosing Python or C#. Flask was again the natural choice as it was used in the curriculum at the University of Stavanger. It's a lightweight framework written in Python that is simple and flexible. However, for this project, several factors made .NET the more appropriate choice.

The most significant limitation of Flask is concurrency as it is a synchronous web

framework by design. Another advantage of .NET is C# which is a statically typed language which unlike Python shows errors at compile time. .NET also offers higher performance because it uses compiled code instead of interpreted code [13]. Additionally SignalR is a first party .NET library which integrates naturally into the .NET backend without any additional concerns [12]. The last reason as to why .NET was chosen was because it is widely used in the industry around the University of Stavanger which makes this project good practice for future work.

2.2.3 Containerization of the application

A consistent execution environment is required to ensure that the application behaves the same way regardless of where it is started. To achieve this the application must be deployed with all dependencies and configuration files. According to [14] there are two ways to solve this, virtual machines or containers. The traditional solution used to be virtual machines. A virtual machine emulates a real computer on top of a physical host machine. The problem with traditional virtual machines is the overhead with each machine having its own guest operating system. This is why containerization was chosen, because it addresses the problem using lightweight containers that use the host operating system instead of multiple guest operating systems.

For containerization there are multiple options to choose from, for example Podman or Docker. According to [15] both Docker and Podman implement the Open Container Initiative (OCI) standards for container images and runtimes, this means that they can get similar results from the same container images. The key difference between lies in the architecture. Docker uses a daemon-based architecture, where a background daemon manages all the containers. Podman on the other hand uses a daemonless architecture where containers are child processes directly managed by the Podman CLI. This makes for a simpler architecture, and gets rid of the daemon as a single point of failure.

Despite Podman's architectural advantages, Docker was chosen for this project for two reasons. First, the Docker ecosystem and CLI are far more mature with virtually all tools having built-in support for Docker making it the default choice in most developer environments [15]. Second, Docker has been the team's

primary containerization tool through prior project experience, and its near-universal adoption in the industry means that virtually all tooling and documentation assumes Docker by default, making it the more practical choice for this project.

2.3 Data Management

Telemetry data is continuously generated throughout a rocket flight, placing strict requirements on how it is stored and retrieved. This section covers the data management approach used in this project.

2.3.1 Telemetry as Time-Series Data

Telemetry data consists of values recorded over time, where each data point represents a measurement (e.g., altitude, temperature, or battery voltage) paired with a precise timestamp. This type of data is categorized as time-series data. Unlike traditional data models, the primary axis of organization in a telemetry system is time, and the workload is characterized by heavy write operations of new data points and queries that aggregate data over specific time intervals [16].

In a ground control system, managing this data efficiently is critical for both real-time monitoring and post-flight analysis. High-frequency sensors, such as altitude and temperature sensors, may produce hundreds of samples per second, requiring a storage solution that can handle high ingestion rates without bottlenecking the rest of the system.

2.3.2 Persistent Storage: InfluxDB vs. Relational Databases

To ensure that mission-critical telemetry is preserved for post-flight analysis, the system implements a persistent storage layer. While the backend processes data in real-time, it simultaneously archives every data point to an InfluxDB instance, this happens both locally, and on external servers, allowing the data to be accessed after missions for analysis, testing, and replay.

When selecting a storage solution, a relational database such as MySQL was considered as an alternative. Relational databases are well proven systems with broad support. They are able to store time-series data, but were never designed to optimize for time, they are built for updates, joins, and transactions rather than the high-frequency append-only workloads typical of telemetry systems [17].

InfluxDB, by contrast is a purpose built time-series database, where time is fundamental. Every piece of data is associated with a timestamp and the storage is optimized for high write throughput required by continuously generated sensor data, avoiding bottlenecks during ingestion [18].

For this project, telemetry data arrives continuously during flight and must later be queried over specific time intervals. This is a core requirement of the system, as operators need to be able to replay recorded sessions and perform post-flight analysis. For these reasons, InfluxDB's data model is a natural fit. Persistent storage also supports the replay feature, where previously recorded data can be loaded and visualized in the same interface used for live monitoring.

2.4 System Design Considerations

This section outlines the key design considerations that shaped the architecture of this project, including how components are kept independent and how real-time challenges can be addressed.

2.4.1 Decoupling and Modularity

A key principle in this project is to reduce dependencies between different parts of the system. For example by separating data handling from user interface components, changes can be made without affecting the entire application. Another example is to make interchangeable components in the backend in the form of interfaces so they can be switched without the need for refactoring the whole application.

This is achieved through the use of Redux for state management and a component-based structure in React. Together, these approaches make it easier

to add new telemetry systems, modify existing functionality, and maintain the code over time because a single source of truth is utilized [7].

2.4.2 Challenges in Real-Time Systems

Building a system that handles both live and stored data introduces several challenges. The system must process incoming data continuously while also storing it without affecting performance. In addition, supporting replay functionality requires switching between live data and stored data. This must be handled in a way that does not disrupt the user interface or create inconsistencies in the displayed data. Another challenge is ensuring low latency in data processing and visualization. In a ground control system, even small delays can reduce the usefulness of the data, especially when monitoring critical parameters in real time. This requires efficient data pipelines and optimized communication between the backend and frontend. Data synchronization is also a key concern. Multiple components may depend on the same data streams, and inconsistencies between these components can lead to incorrect interpretations. Ensuring that all parts of the system reflect the same state at any given time is therefore essential. Finally, robustness and fault tolerance must be considered. Network interruptions, delayed data packets, or system failures should not cause the application to crash or behave unpredictably. Instead, the system should gracefully handle such issues, for example by buffering data or providing fallback mechanisms.

2.5 Hosting and Deployment

The sponsor version of the system is intended to be hosted online. For this UiS agreed to provide Unix servers as hosting infrastructure. This ensures the sponsor application runs separately from the launch computer. Hosting on UiS servers also keeps the infrastructure within the university's own environment, which simplifies access, and reduces the need for external hosting services. Because the application is fully containerized with Docker, deploying it to the UiS servers only requires running a single command.

2.6 2025 Thesis

This section discusses the previous ground control system developed in the 2025 bachelor thesis by Sebastian Grønbech and Bogdan-Valentin Raicu [2].

The 2025 thesis represented an important step in the development of ground control software for UiS Aerospace. A clear strength of that work was that it established a functioning technical foundation for ground control by combining a graphical user interface with backend support for real-time communication, telemetry monitoring, and command handling. It also introduced technology choices that were still relevant for this project, such as .NET, React and SignalR. In addition, the previous thesis demonstrated that live telemetry could be decoded, transmitted, and visualized in real time, showing that the basic concept was feasible in practice.

At the same time, the 2025 thesis also had several important limitations. Although the ambition was to deliver a complete ground control application that could support several connected client machines and multiple dashboards, the final result was more limited. The thesis states that one functioning and integrated dashboard was completed, while other dashboard modules were still present in the code but had not been fully connected into a complete operational system.

Another challenge encountered during development was the need to adapt to changing conditions in surrounding subsystems. The software group took on responsibility for CAN decoding when the telecommunication group encountered technical difficulties, which is the kind of cross-group collaboration that is expected in a multidisciplinary project. Delays in integration testing and incomplete signal definitions did however create some bottlenecks that affected how much functionality could be completed within the project period.

The previous thesis also identified broader structural limitations in the earlier ground control solution used in UiS Aerospace. These included weaknesses in security and command execution, poorly optimized visualization, strong coupling to the telecommunication microcontroller, and reliance on local storage on the machine running the GUI. The thesis further noted that this solution was no

longer compatible with the requirements of the Phoenix project. These limitations are important because they show that the need for a new system was not only about improving the interface, but also about creating a solution that was easier to maintain, easier to adapt, and better suited for future missions.

For this reason, the present thesis builds on the ideas and experience from the 2025 work, but not by simply continuing the same system unchanged. Several of the technology choices were retained, but the architecture was redesigned to better support the needs of the current project. In particular, this thesis places greater emphasis on modularity, persistent telemetry storage on UiS servers, replay of recorded sessions, and streaming of the latest launch data to external viewers. The decision to develop a new ground control system was therefore motivated by both the strengths and weaknesses of the previous thesis: it provided a valuable technical foundation, but a more flexible, maintainable, and extensible solution was needed for the continued development of UiS Aerospace.

Chapter 3

System Design and Implementation

This chapter describes the design and implementation of the ground control system developed in this thesis. While Chapter 2 presented the relevant concepts and technologies, this chapter explains how the system itself was structured and implemented.

3.1 System Overview

The system was designed as a distributed application with three main parts: a backend, a database, and a frontend. The frontend and the backend were split into two parts, the operator and the sponsor versions which serve different roles. The system was also designed for two databases for redundancy.

The operator interface is intended for internal use during testing, launch and post-launch for analysis. It provides live telemetry, status information, command functionality, and replay functionality for the launch team. In addition, telemetry data is written to both the offline and cloud databases, with the cloud data being available through a separate public website hosted on external servers. The sponsor interface is accessible from a public website allowing sponsors and other observers to follow the launch in real time. The sponsor interface is only intended

for viewing the latest live launch and does not have access to internal operational features that are included in the operator interface.

This architectural separation serves a dual purpose. It safeguards critical operations by isolating them from external viewers, while providing access to mission data without exposing internal control functions. Since the systems are isolated an increase in viewers will not affect the load on the operator interface and thus not interfere with the ground control.

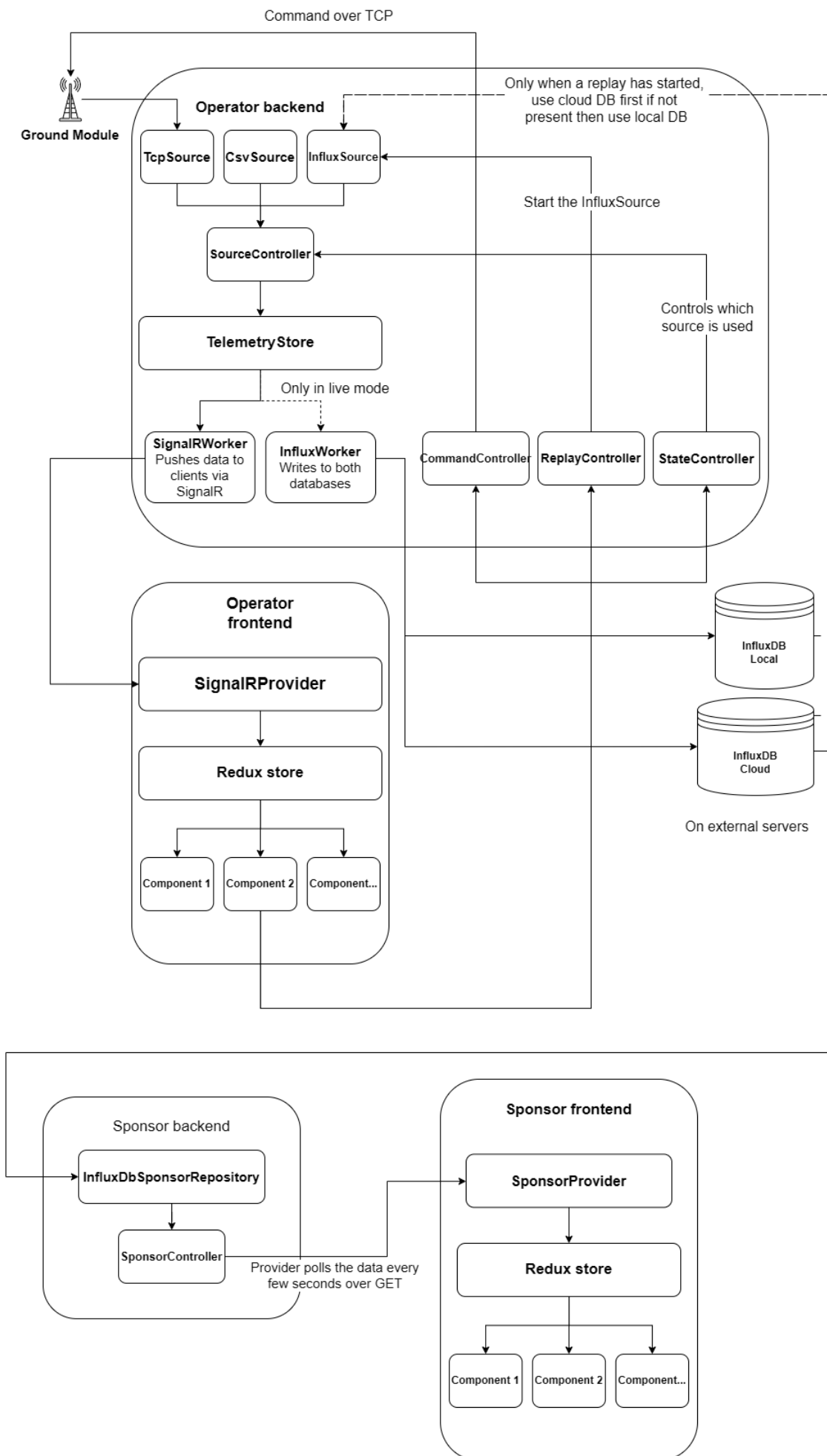


Figure 3.1: System architecture overview showing the operator subsystem (top) and sponsor subsystem (bottom).

3.2 Architecture

The operator system is structured around a flow of data from the telemetry input where the data is then processed and added to a store. This store then dispatches an event causing the data to get written to the databases and sent out via SignalR to the operator frontend as seen in Listings 3.1 and 3.2. Using this approach workers may subscribe to the event independently. Stored telemetry can later be used for replay and post-flight analysis. The telemetry is stored both offline and in the cloud for redundancy with offline being used as a fallback solution when the cloud is down.

Listing 3.1: TelemetryStore event dispatch

```
1 public void Add(SensorData item)
2 {
3     _data.Enqueue(item);
4     OnUpdate?.Invoke(item);
5 }
```

Listing 3.2: Workers subscribing to TelemetryStore simplified

```
1 // SignalRWorker
2 _store.OnUpdate += async (data) =>
3 {
4     await _hubContext.Clients.All.SendAsync("update", data,
5         stoppingToken);
6 };
7 // InfluxWorker
8 _store.OnUpdate += async (data) =>
9 {
10     await _localRepo.WriteAsync(data);
11     await _cloudRepo.WriteAsync(data);
12 };
```

The sponsor system is structured around polling from the frontend. This polling fetches the newest data every few seconds, polling was chosen over SignalR to keep traffic low to the server because of the increased amount of users. The data is fetched from the sponsor backend that again fetches from the cloud database.

This separation also helps keep the system easier to maintain, and less access control is required inside the backend than if the system had been combined.

3.3 Backend Implementation

The backend works by getting the telemetry data from the input source, processing it, saving it to the database then sending it to the client. There are some differences between the operator and sponsor versions, but since both are built from the same code they are similar in how they operate. The system groups data by sessions and has data sources which can be interchanged, the current implementation has a TCP, InfluxDB, and CSV source. When using the TCP and CSV sources, the backend state is set to live, the InfluxDB source is only used when the state is set to replay, the source switching is shown in Listing 3.3. The source controller starts the new source using a semaphore to prevent race conditions. These sources all process the data input their own way before feeding the data into the store, the store then dispatches a new OnUpdate event. The InfluxWorker then catches this event if the internal state is set live, this is so that the data gets stored in the databases. At the same time the SignalRWorker regardless of the backend state also catches the event and pushes the data to the client.

For live operations the TcpSource is used because the data sent from the ground module will be sent over TCP, if the state is set to replay the application uses the InfluxSource to fetch all session data from the database. The last source type is the CsvSource which was used during early testing with previous datasets. The application has an API that exposes several endpoints used for controlling, commands, state, replays. The commands endpoint takes a post request with a command id, the backend then validates a command key header to check that the appropriate password is set before it sends a command over TCP to the ground module. The state endpoint is used to transition between idle, live, and replay states. Lastly the replay controller is used to fetch all replays. The sponsor version of the backend does mostly the same but the only source is the current session in the cloud database. It has a sponsor API endpoint which either returns all session data, or just X amount of seconds back with data. Because both versions are built from the same code, attributes were added to the API controllers

and only inject the dependencies that are necessary, therefore it is not possible to send commands, change states, or start replays from the sponsor versions. To handle the configuration of the different IP-addresses for TCP, database connection details, and command key, these parameters are defined using environment variables provided through the docker-compose file. This ensures that the sponsor and operator builds are separate unique containers, and ensures easy configuration. An example docker configuration is shown in Listing 3.4.

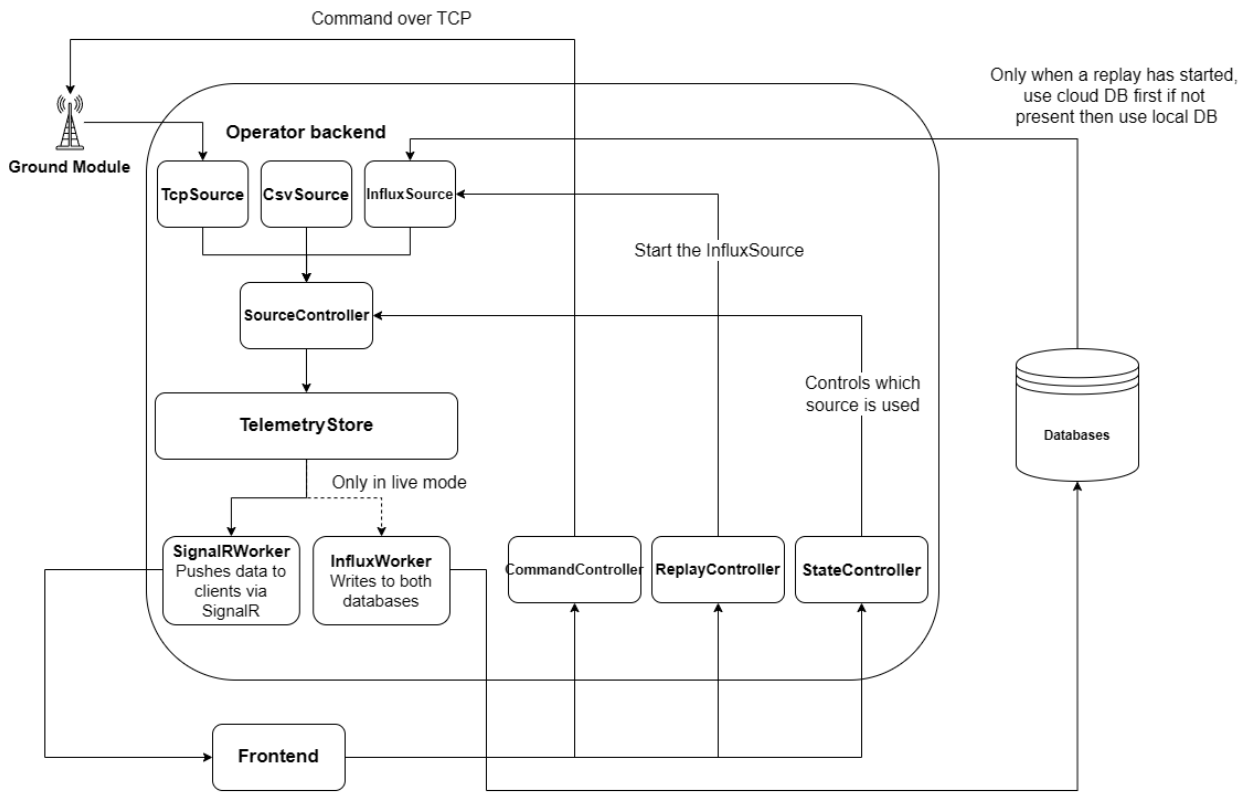


Figure 3.2: Backend architecture overview for the operator version. The diagram shows the flow of data between the backend, ground module, databases and the frontend.

Listing 3.3: SourceController switching between data sources

```
1 public async Task SwitchTo(DataSource sourceType)
2 {
3     await _lock.WaitAsync();
4
5     try
6     {
7         if (_currentSource?.Type == sourceType)
8             return;
9
10        if (_currentSource != null)
11        {
12            await _currentSource.StopAsync(CancellationTokens.None);
13            _currentSource = null;
14        }
15
16        var newSource = _sources.FirstOrDefault(source => source
17            .Type == sourceType);
18
19        if (newSource != null)
20        {
21            _store.Clear();
22            await newSource.StartAsync(CancellationTokens.None);
23            _currentSource = newSource;
24        }
25    }
26    finally
27    {
28        _lock.Release();
29    }
```

Listing 3.4: Environment variables in Docker Compose for operator mode

```
1 api:
2   environment:
3     - AppMode=Operator
4     - CommandServerIP=host.docker.internal
5     - CommandServerPort=8090
6     - CommandKey=1a2b3c
7     - InfluxDBLocal__Url=http://phoenix_influx_local:8086
8     - InfluxDBLocal__Token=my-super-secret-local-token
9     - InfluxDBCloud__Url=http://cloud-ip:8086
10    - InfluxDBCloud__Token=my-super-secret-cloud-token
```

3.4 Frontend Implementation

The frontend was designed as a dashboard intended for internal use during testing, launch, and post-flight analysis. Its purpose is to present telemetry data in a form that is easy to interpret while enabling operators to monitor multiple values simultaneously. In addition to telemetry visualization, the operator version provides functionality for state control, replay execution, and command transmission.

The frontend is built around reusable interface components. Rather than implementing each display element as a separate custom solution, the system uses a common set of building blocks for charts, value displays, status indicators, and other telemetry views. These components are connected to telemetry values through identifiers, which makes it possible to reuse the same component for different signals.

This design makes the interface easier to extend. When new telemetry values are introduced, existing components can often be reused instead of rewritten. Supporting new sensors therefore mainly requires changes to configuration and layout rather than modifications to the underlying interface logic, minimizing the amount of work needed to configure the project for a new mission. Because of this modularity, the project could easily be altered for uses other than just rocket launches.

The frontend also separates presentation from data handling. Components are responsible for displaying values, while the underlying telemetry data is managed through shared application state as shown in Figure 3.3. Depending on whether the system is running the operator or sponsor version, data is retrieved either through SignalR or through polling. Once received, the data is stored in the Redux store, which is responsible for updating the application state. This keeps the interface logic simpler and improves maintainability by ensuring that shared state is managed in one central location.

Because several parts of the dashboard depend on the same incoming telemetry data, a shared state model was adopted in the frontend. Live values received from the backend are stored centrally in the Redux store and accessed by the components that require them. Telemetry values are retrieved by identifiers such as `Altitude` and `Velocity`, after which a component may either access the latest value only or the complete time series, depending on what it needs to display. This is illustrated in Listing 3.5 and Listing 3.6.

This approach avoids prop drilling and reduces coupling between different parts of the interface. A chart component, for example, only needs access to the relevant telemetry data in the shared state and does not need to manage how the data is received from the backend. Listing 3.5 shows how a chart can be rendered by providing only a telemetry identifier as an argument, while Listing 3.6 shows how the chart component retrieves the corresponding data directly from the Redux store.

The main advantage of this structure is that it makes the interface easier to modify and extend. To display a new telemetry signal in an existing chart component, it is generally sufficient to provide a different identifier through the `sensorId` argument. New views can therefore be introduced without changing the internal logic of existing components, as long as they can access the required telemetry data through the shared state. In addition, the Redux store ensures that all connected components remain synchronized, so that the interface consistently reflects the current system state.

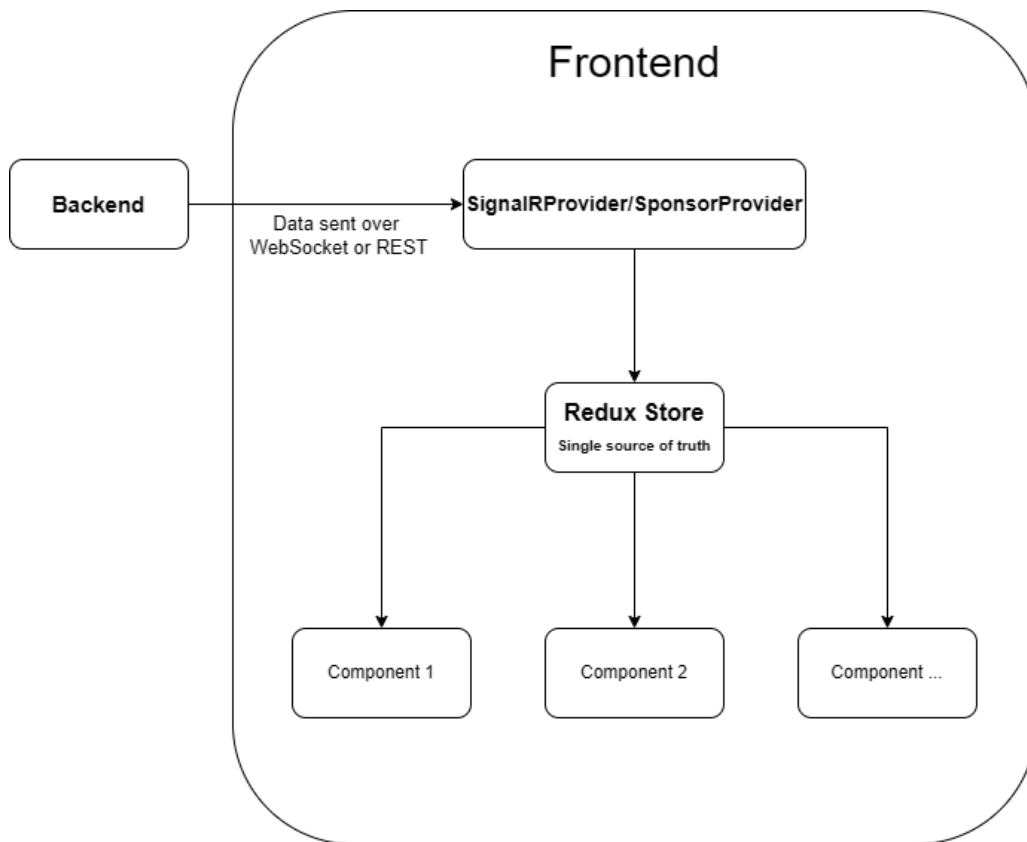


Figure 3.3: Frontend architecture overview showing the flow of data from the backend to the components.

Listing 3.5: Mission archive page rendering a telemetry chart

```
1 import TestChart from "../components/TestChart";
2
3 export default function mission_archive() {
4   return (
5     <div>
6       <TestChart sensorId="Altitude" />
7     </div>
8   );
9 }
```

Listing 3.6: Simplified telemetry chart component

```
1 import { LineChart, Line, XAxis, YAxis, Tooltip,  
    ResponsiveContainer } from "recharts";  
2 import { useSelector } from "react-redux";  
3 import { selectTelemetryValuesById } from "../state/telemetry/  
    telemetrySlice";  
4  
5 export default function TestChart({ sensorId }) {  
6     const data = useSelector(selectTelemetryValuesById(sensorId)  
        );  
7  
8     return (  
9         <ResponsiveContainer width="100%" height={300}>  
10             <LineChart data={data}>  
11                 <XAxis  
12                     dataKey="timeStamp"  
13                     tickFormatter={(time) => new Date(time).  
                        toLocaleTimeString()}  
14                 />  
15                 <YAxis />  
16                 <Tooltip  
17                     labelFormatter={(time) => new Date(time).  
                        toLocaleTimeString()}  
18                 />  
19                 <Line  
20                     type="monotone"  
21                     dataKey="value"  
22                     dot={false}  
23                     isAnimationActive={false}  
24                 />  
25             </LineChart>  
26         </ResponsiveContainer>  
27     );  
28 }
```

3.5 Telemetry Storage and Replay

A central part of the system is the persistent storage of telemetry data. During live operation, sensor values are not only displayed to connected clients, but are also written to storage locally and on external servers. This ensures that the data remains available after testing and launch for post-flight analysis.

Persistent storage makes it possible to review missions after they are completed. Stored telemetry can be used to inspect system behaviour, compare measurements, and support later analysis. It also reduces dependence on the local machine used during a launch, since the data is not limited to a single environment.

The same stored telemetry is also used to support replay. Telemetry is stored by sessions and in replay mode, previously recorded sessions can be loaded and displayed through the operator interface the same way it was displayed live. This makes it possible to test the dashboard using recorded data and to examine mission events after flight using the same type of interface as in live operation.

3.6 Public Website

In addition to the internal operator interface, the project includes a public-facing website which runs the sponsor interface. The purpose of this part of the system is to present the latest telemetry data to sponsors and other observers during missions.

This website is separate from the internal control interface. It does not include command functionality or the full set of operational tools used by the launch team. Instead, it focuses on presenting relevant launch information in a simple and readable form. This allows external viewers to follow the mission without access to features intended only for operators. The website also fetches data from a sponsor version of the backend keeping it from congesting the traffic to the operator backend. This version of the interface uses polling instead of SignalR causing the overall traffic to be less than if it used SignalR, but at the cost of some small delay between data fetches.

By separating the public website from the internal dashboard, the system can support both mission operations and external viewers. The operator interface remains focused on monitoring and control, while the public website serves as a way to share the launch with a wider audience.

3.7 Security

Since the ground control system is designed to operate locally on a single machine at the launch site, the network security was built around restricting external access rather than implementing authentication between the individual services.

In the operator configuration, the API port is explicitly bound to 127.0.0.1 rather than the default 0.0.0.0. This means that the backend only will be reachable locally on the same machine, and no other device can connect to the API directly. This choice was made deliberately considering that the ground station machine never should be exposed to an external network during a launch.

The backend has also implemented a mode based access control which is based on what mode the backend application is started in. This access control is enforced through two custom ASP.NET action constraints `OperatorOnlyAttribute` and `SponsorOnlyAttribute`, these are applied at the controller level as seen in Listing 3.7. When the backend starts in sponsor mode all controllers decorated with `OperatorOnlyAttribute` will be physically unavailable. Instead of the routes simply returning a 403 Forbidden code, the endpoints will not be registered at all.

Listing 3.7: Applying mode based access control to an API controller

```
1 [OperatorOnly] // When this decorator is applied the controller
   gets restricted to only the operator mode
2 [CommandKey]
3 [ApiController]
4 [Route("api/[controller]")]
5 public class CommandController : ControllerBase
6 {
```

As sending commands to the rocket is the most safety-critical action in the system,

it has to be protected by an additional layer of security beyond the operator mode check. `CommandController` requires a secret key to be present in the `X-Operator-Command-Key` HTTP header on every request. The key is configured in the backend and must be entered by the operator in the frontend before any commands can be sent. If the key is missing or incorrect the backend returns a 401 `Unauthorized` response instead of forwarding the command to the rocket.

Input validation is also implemented at both layers. The frontend uses a regular expression that ensures that only integers may be entered in the command field. The backend also checks this using the `int.TryParse` function to make sure no malformed or non-integer command values are sent to the rocket.

3.8 Command Handling

The system includes support for sending commands from the ground station to the rocket or associated subsystems. This is part of the operator interface and is intended for use during testing and mission operations. The interface asks for confirmation before sending a command since they affect critical systems. Because of this, access to this functionality is restricted. The implementation therefore includes password protection before commands can be issued. This reduces the chance of accidental or unauthorized use. The password is provided as a header on a POST request to the operator backend. When sending a command, it and its response will be logged in the console.

At the same time, the command interface was designed to allow more than a fixed set of predefined actions. Operators can also enter custom commands when needed. This makes the system usable in different testing situations and avoids having to redesign the command interface every time a new command is introduced.

3.9 Deployment and Development Setup

The different parts of the system were developed and tested as separate services. This made it easier to run the frontend, backend, and database in a controlled way during development.

Container-based deployment with Docker was used to simplify setup across different machines. This reduced configuration differences between development environments and made it easier to start the required services consistently. This was especially useful in a student project setting, where several developers could work on the same system using different local machines. The final setup also supports deployment of the database and sponsor interface on external servers, which is an important part of making telemetry available during launch and presenting data online to external viewers.

3.10 Chapter Summary

This chapter has described the design and implementation of the ground control system developed in this thesis. The system is structured around telemetry handling, persistent storage offline, and on external servers, an internal operator dashboard, and a separate sponsor dashboard for external viewers.

Together, these parts support live monitoring, telemetry storage, replay of recorded data, command handling, and presentation of current mission data to sponsors and other observers.

Chapter 4

Results

4.1 Experimental Setup and Data Set

To test the application, a Python script was developed, the script simulated the transmission of telemetry data over TCP. The script used telemetry data from a previous launch as input and allowed the delay between each transmitted data line to be configured. This made it possible to test different levels of data throughput and verify that the system handled incoming data correctly.

During testing, no data loss or performance issues were observed regarding the TCP data transmission. Earlier in the project, a similar Python script was used to send data to the backend by writing to a CSV file instead of transmitting it over TCP.

The final TCP-based sender script is shown below:

Listing 4.1: Python script used to simulate telemetry data transmission over TCP.

```
1 INPUT_FILE = Path("data_rewritten_fuel_sparse.csv")
2 SERVER_IP = "127.0.0.1"
3 PORT = 8088
4 DELAY = 0
5
6 def get_current_timestamp():
7     return datetime.now().astimezone().isoformat()
```

```
8
9 def run_sender():
10     if not INPUT_FILE.exists():
11         print(f"Error: Could not find {INPUT_FILE}")
12         return
13
14     print(f"Connecting to C# server at {SERVER_IP}:{PORT}...")
15     try:
16         with socket.socket(socket.AF_INET, socket.SOCK_STREAM)
17             as s:
18             s.connect((SERVER_IP, PORT))
19
20             with INPUT_FILE.open("r", newline="", encoding="utf
21                 -8") as infile:
22                 reader = csv.reader(infile, delimiter=";")
23                 header = next(reader) # Skip header
24                 for row in reader:
25                     if not row:
26                         continue
27                     row[0] = get_current_timestamp()
28                     message = ";".join(row) + "\n"
29                     s.sendall(message.encode("utf-8"))
30                     time.sleep(Delay)
31             print("Finished sending all data.")
32
33     except ConnectionRefusedError:
34         print("Could not connect to C# server. Is the server
35             running?")
36
37     except Exception as e:
38         print(f"An error occurred: {e}")
39
40 if __name__ == "__main__":
41     run_sender()
```

4.2 Integration testing with the telemetry group

As part of the experimental evaluation, the system was planned to be tested with the ground module developed by the telemetry group at UiS Aerospace. This would have allowed for a full end-to-end integration test, validating that the systems could receive and process real telemetry data from actual hardware in real time over TCP.

However during the course of the project, UiS Aerospace decided to postpone the rocket launch to 2027. As a consequence of this decision, the telemetry group deprioritized completing their ground module as the deadline had been set back. This meant that no physical hardware was available for integration tests before the submission deadline.

As a result, all testing was carried out using the Python script described in section 4.1. This test validated that the system handles data correctly under realistic conditions, but does not replace a full hardware integration test. Because of this a complete integration test with the ground module remains as future work, and is expected to be carried out closer to the 2027 launch.

4.3 Testing of the sponsor functionality

Early during development, testing of the sponsor functionality was carried out over two machines. One was the operator machine using the earlier mentioned Python script 4.1 to simulate the ground module, and the other was the sponsor machine that also hosted the cloud database. This test was also a success and the operator backend managed to write the data to both the local and cloud databases. The sponsor frontend also successfully showed the live launch data without issues.

The next step was to validate that the sponsor application could be deployed to an external server. As the UiS Unix servers had some permission problems at the time of testing, an external hosting provider was used as a test environment instead of the UiS Unix servers. The sponsor application was deployed with the existing sponsor configuration file, and the frontend was confirmed to be working

while deployed. This test validated that the containerized deployment works correctly in a cloud environment. The same approach will be used when deploying to the UiS Unix servers at a later time.

4.4 Dashboards

As part of the frontend, four dashboards were developed to support the operational roles within UiS Aerospace. These consisted of one main dashboard and three subsystem-specific dashboards for propulsion, avionics, and recovery. The main dashboard contained only the command console, providing operators with direct access to command functionality without unnecessary visual distractions.

The dashboards were developed through an iterative design process. First, an Excel overview sheet was distributed to the relevant groups, as shown in Appendix C. In this sheet, the groups could specify which telemetry values, subsystem states, and visualizations they considered most important. This made it possible to collect requirements directly from the intended users and helped ensure that the dashboards reflected operational needs rather than only technical assumptions.

Based on the collected input, a set of mock-ups was created to explore layout alternatives, grouping of related variables, and visual prioritization. These mock-ups were then discussed and refined before the final dashboards were implemented in the frontend. After implementation, the dashboards were presented to the corresponding groups for approval.

4.4.1 Main Dashboard

The main dashboard contains only the command console, as shown in Figure 4.1. This provided operators with direct access to command functionality during testing and mission operations without unnecessary distraction from other interface elements. To prevent unauthorized access, this module was password protected and only available when the system was running in operator mode.

The command console, shown in Figure 4.2, contains three command buttons: *Launch*, *Abort*, and *Turn Off Engine*. These buttons represent critical actions during launch operations. To reduce the risk of accidental activation, a confirmation prompt is shown when one of the command buttons is clicked, as shown in Figure 4.3. The command is only sent after the operator confirms the action.

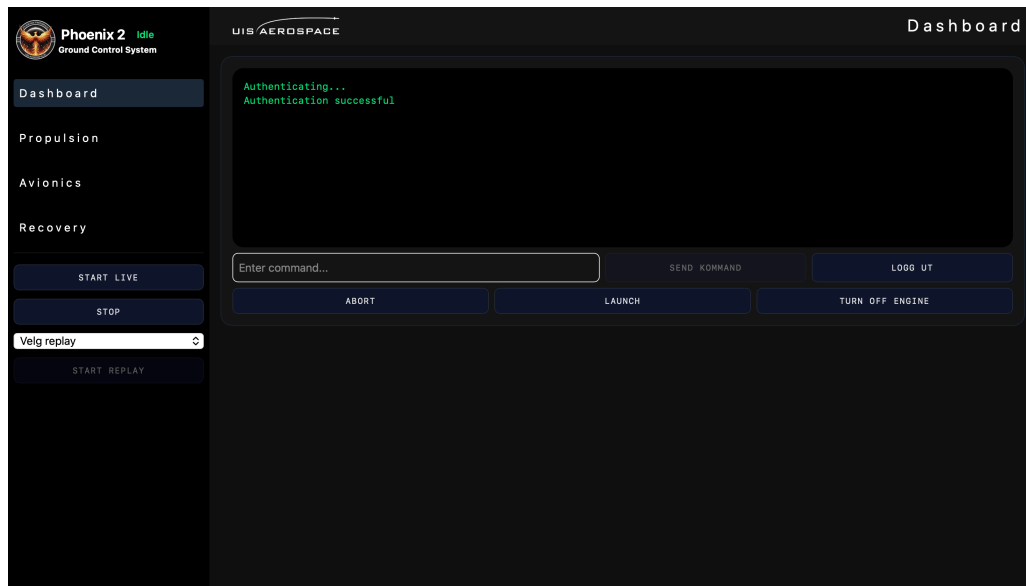


Figure 4.1: The main dashboard after successful authentication, showing the command console with the full navigation sidebar and session controls. Start Live, Stop, and Start Replay controls are accessible from the sidebar regardless of which dashboard is active.

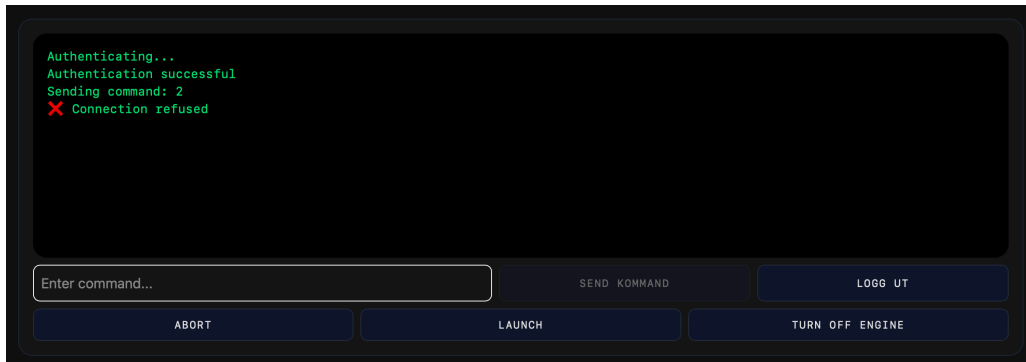


Figure 4.2: The command terminal displaying the results of commands sent to the rocket, with colour-coded feedback indicating authentication and command status. Quick-access buttons for Abort, Launch, and Turn Off Engine are available below the input field.

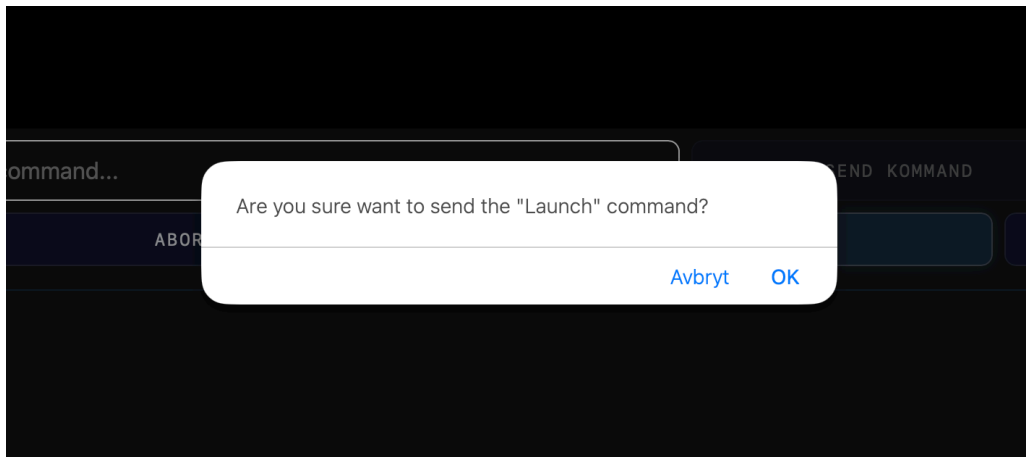


Figure 4.3: Confirmation dialog shown when a critical command such as Launch is triggered, requiring the operator to explicitly confirm before the command is sent to the rocket.

4.4.2 Propulsion

The propulsion dashboard was designed to support monitoring of fuel-related and engine-related behaviour. In particular, the propulsion group emphasized the need to monitor fuel consumption, making this one of the most important considerations in the design of the dashboard. The view therefore prioritized telemetry related to tank levels, pressures, and temperature measurements relevant to propulsion system performance.

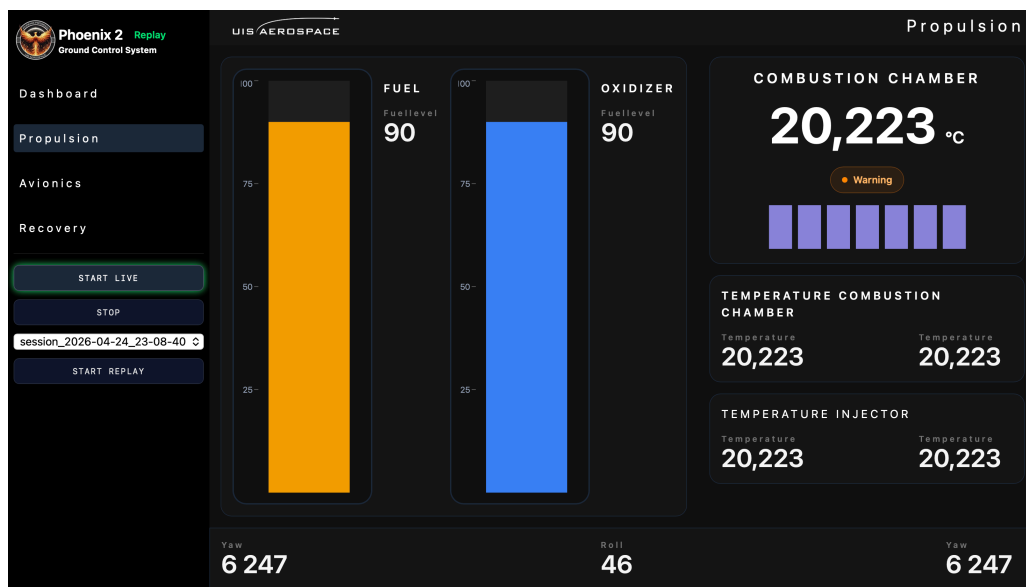


Figure 4.4: The propulsion dashboard displaying fuel and oxidizer tank levels, combustion chamber temperature and status, and injector temperatures. The layout groups engine and tank data to give the propulsion team a quick overview of system health.

4.4.3 Avionics

The avionics dashboard was designed to present the information most relevant to the rocket's motion and position. According to the needs expressed by the avionics group, particular emphasis was placed on height, direction, and

position. In addition to numerical and graphical telemetry displays, the dashboard also included a three-dimensional rocket model to provide a more intuitive representation of the rocket's orientation and movement.

The 3D rocket model was created internally in UiS Aerospace and integrated into the dashboard based on a request from the avionics team. This made it possible to display the rocket visually alongside the telemetry values, giving operators a clearer understanding of the rocket's orientation during testing and replay.

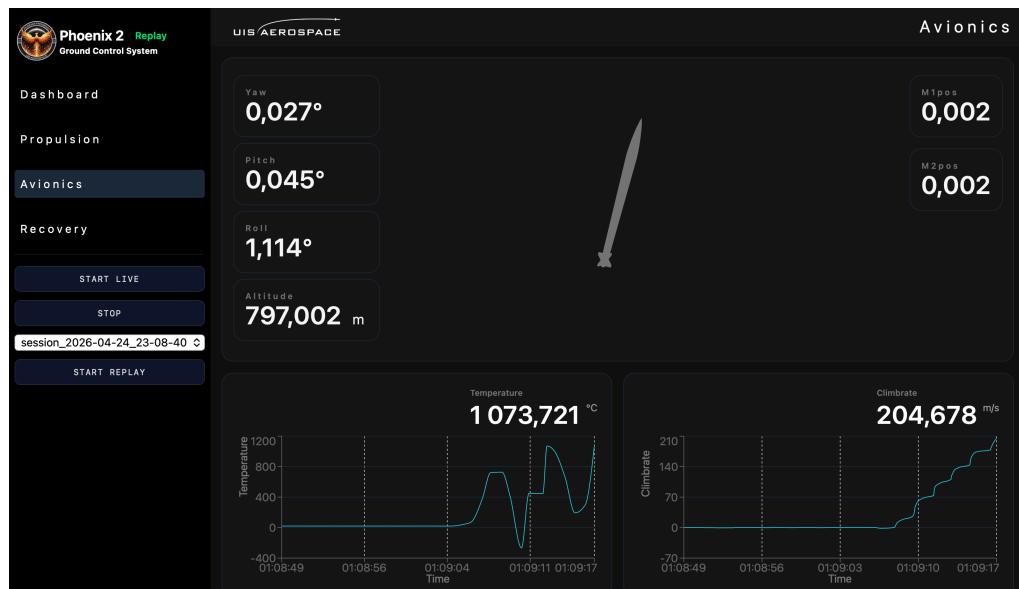


Figure 4.5: The avionics dashboard showing yaw, pitch, roll, and altitude readings alongside a 3D rocket model of the Phoenix 2, provided by UiS Aerospace, reflecting the rocket's current orientation. Temperature and climb rate are shown as time-series charts in the lower section.

4.4.4 Recovery

The recovery dashboard was intended to provide the recovery group with a clear overview of subsystem status and mission phase. Rather than focusing primarily on continuously changing sensor values, this dashboard emphasized status information and the current phase of operation. This made it easier for the group

to assess whether the system was in the expected state and whether recovery-related events had progressed as intended.

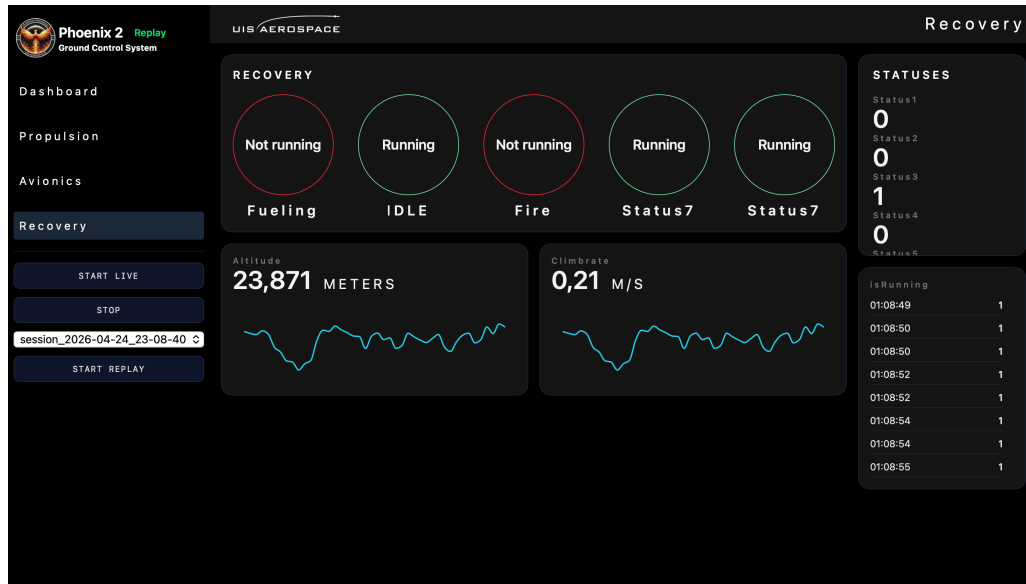


Figure 4.6: The recovery dashboard showing mission phase indicators (Fueling, IDLE, Fire) as running/not-running circles, with altitude and climb rate charts below. A timestamped status log is visible on the right side panel.

4.4.5 Start, Stop, and Replay Module

The start, stop, and replay functions were implemented as part of the navigation bar so that they remained easily accessible within the operator interface regardless of which dashboard was currently in use. Placing these controls in a persistent location allowed the operator to change the system state without leaving the active view, which simplifies workflow during testing, launch and analysis.

These functions were restricted to operator mode and were not available in the sponsor version. This was an important design decision, since allowing external users to start, stop, or replay the system could open for unintended security flaws. Limiting access to a single operator therefore reduced the risk of

misuse and helped preserve system stability.

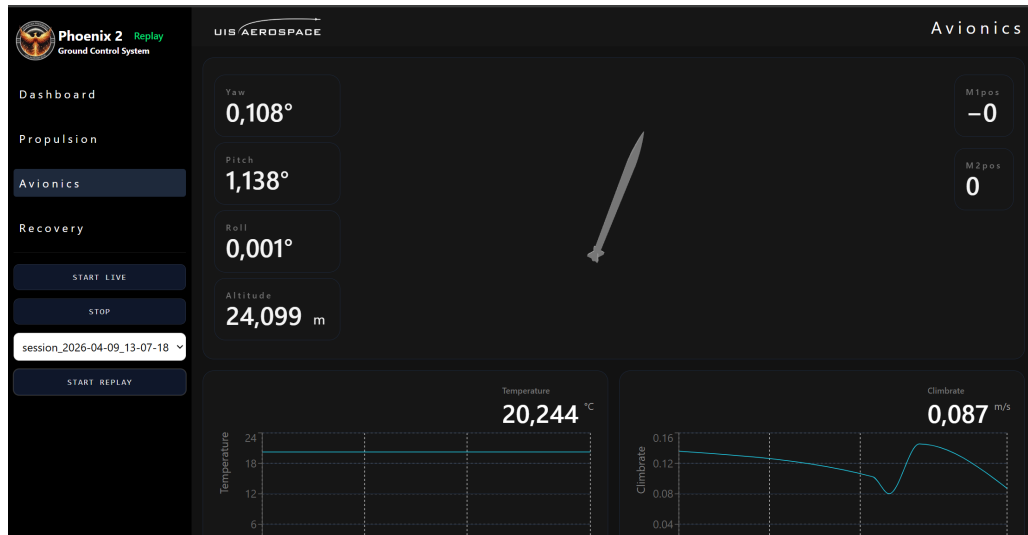


Figure 4.7: The avionics dashboard in replay mode, indicated by the "Replay" label in the top-left header.

Chapter 5

Discussion

5.1 Comparing with previous project

The previous ground control system developed by Grønbech and Raicu [2] identified several limitations that this project aimed to address. The most significant of these were the lack of persistent cloud storage, and a tightly coupled architecture that made it difficult to adapt to new telemetry structures.

These limitations have been resolved in this project. Telemetry data is now stored persistently in a cloud database on external servers, providing redundancy that was previously missing. The tight coupling between data handling and interface components has been replaced by a Redux-based state management approach and a component-based frontend. Because of this, new telemetry sources or visualizations can be added without requiring changes across the entire application. In addition, this project introduced a sponsor-facing web application that allows external observers to follow live telemetry data without compromising operator security. A replay functionality was also added, enabling post-flight review of recorded telemetry sessions.

Whether these improvements represent a genuine advancement is not straightforward to assess in isolation. The architectural changes, namely Redux state management, component separation, and persistent cloud storage, provide measurable structural benefits: the codebase is more modular, easier to extend,

and no longer reliant on local storage. The delivery of multiple completed dashboards, covering propulsion, avionics, recovery, and a main overview, represents a step forward in operational coverage compared to the single functioning dashboard delivered by the previous thesis. The sponsor application and replay module are entirely new capabilities with no equivalent in the previous system. In that regard, the solution is objectively broader in scope and more complete as a ground control platform for launch use.

However, it is important to acknowledge areas where the previous project had relative strengths. Grønbech and Raicu were the first to establish the foundational technical stack, including .NET, React, SignalR, and InfluxDB, and demonstrated that live telemetry decoding and real-time visualization were feasible end-to-end. That groundwork reduced the unknowns significantly for this project.

Furthermore, the previous thesis took on CAN decoding responsibilities in addition to GUI development, a contribution that extended beyond the scope of the ground control interface itself. This project benefited from a narrower and more clearly defined scope, which made it more feasible to deliver a complete and integrated result. Had the previous group operated under the same focused scope from the start, their outcome may well have been comparable.

In summary, this project builds meaningfully on the previous work and delivers a more complete, maintainable, and extensible ground control system. This progress was made possible precisely because the previous thesis laid the necessary technical foundation. The improvements should therefore be understood as an evolution of that work rather than a replacement of it.

5.2 Working with the other groups and launch postponement

Collaboration with other groups within UiS Aerospace was primarily focused on the dashboard design process. Through the Excel overview sheet described in section 4.4, the propulsion, avionics, and recovery groups were able to specify which telemetry values and visualizations were the most relevant to their work.

This feedback was reflected in the final dashboard designs, and all groups approved the result.

The postponement of the rocket launch to 2027 was the biggest hurdle for this project. The biggest consequence was that the telemetry group deprioritized completing their ground module, this meant that a full hardware integration test was not possible before the submission deadline. The postponement also meant that the exact format and structure of incoming telemetry and outgoing commands was not fully defined. To account for this uncertainty, the system was designed to receive data and send commands over TCP in a general format, so that future teams can adapt the input and output layers of the application without needing to restructure the whole application.

5.3 Result Evaluation

Overall, the result of the project is considered to be a success. Both the operator and sponsor applications functioned as intended throughout the testing phase. The live data pipeline from the Python script through the backend and out to both the operator and sponsor frontends worked reliably with no observed data loss or performance issues. Seeing live telemetry appear on the sponsor frontend in real time while the operator system was running was a clear demonstration that the core architecture worked as designed.

The sponsor application in particular exceeded initial expectations. The ability to stream live telemetry data from the operator backend to the cloud database, and have it appear on the sponsor frontend with only a short polling delay, validated the two-database architecture and demonstrated that the system is ready for use in a real world scenario.

If the project were to be started over, more time would have been invested in defining a clear interface for communication between the application and the ground module from the telemetry group, to reduce uncertainty around the TCP format. This would have made it easier to test the system against more realistic data structures during development.

5.4 Future Improvements

Several potential improvements were identified during the duration of this project that were not implemented due to time constraints.

The most significant of these is a full integration test with the actual ground module developed by the telemetry group, this remains the highest priority for future work and is expected to be carried out before the 2027 launch.

The replay functionality could also be extended to better support post-flight analysis. The current implementation plays the replay at the same speed as the data was recorded at. This could be extended to have variable speed, the ability to zoom into the specific graphs and add pause/play functionality. Another feature could be playing only specific parts of a replay, and exporting the telemetry data to a downloadable format for further processing.

Video feed streaming was also considered during the project. Integrating a live video feed from the rocket into the ground control interface would give operators increased situational awareness during launch, but this was deprioritized when it became uncertain if the rocket would provide a live feed or have cameras at all.

Finally the current command authentication relies on a shared key configured through environment variables. A more robust solution for future iterations would be to replace this with proper authentication, which would provide for better auditability and make it easier to manage multiple operators.

Chapter 6

Conclusion

This thesis has presented the design, implementation, and testing of a ground control system for the UiS Aerospace suborbital rocket. The work was guided by five specific objectives outlined in Chapter 1.6, each of which has been addressed in the final system.

The first objective was to ensure reliable real-time handling of telemetry data during missions. This objective has not yet been fully validated against live telemetry data. However, internal testing using simulated TCP telemetry showed no data loss or performance issues under realistic conditions, suggesting that the SignalR-based implementation is a promising solution. Full verification will require testing against actual mission telemetry.

The second objective was to store telemetry data persistently on UiS servers. The system writes all incoming telemetry to both a local InfluxDB instance and a cloud-hosted instance on external servers, ensuring that mission data remains accessible after testing and launch, and reducing dependence on any single machine.

The third objective was to enable replay and analysis of recorded flight data. The replay functionality allows previously recorded sessions to be loaded and replayed through the same operator interface used during live operations, providing a consistent environment for post-flight analysis.

The fourth objective was to provide visualization tools that are reusable and adaptable to different mission needs. The frontend was built around a component-based architecture using React and Redux, where telemetry components are connected to data through identifiers rather than fixed signal definitions. This makes it straightforward to adapt the dashboards to new sensors or mission profiles without rewriting the underlying interface logic.

The fifth objective was to make launch data available on a website for sponsors and other observers. This was fulfilled through a separate sponsor frontend, hosted on external servers, which presents live telemetry data through a polling-based connection to a dedicated sponsor backend. The system was successfully tested across multiple machines, with live data appearing on the sponsor frontend in real time.

Although a full hardware integration test with the ground module was not possible due to the postponement of the launch to 2027, the system has been validated through simulated telemetry testing and is considered ready for operational use. The remaining integration work is the highest priority before the 2027 launch, and the system has been deliberately designed to minimize the effort required to complete it. The deployment of the system in a real mission environment during the launch of the Phoenix 2 rocket represents the final validation of this work.

Bibliography

- [1] UiS Aerospace. “Phoenix 1 aftermovie.” Published 24 August 2025. Accessed April 2026.
- [2] S. Grønbech and B.-V. Raicu, *Bakkestasjonen applikasjon for romrakett*, Bachelor Thesis, UiS Aerospace, 2025. [Online]. Available: <https://nva.sikt.no/registration/019903f1cd10-13e75509-7d33-46b4-bfea-db0dced4e074>.
- [3] T. Singh. “Javascript frameworks in 2024: React vs. vue vs. svelte – which one to choose?” DEV Community, Accessed: Apr. 15, 2026. [Online]. Available: <https://dev.to/tarunsinghofficial/javascript-frameworks-in-2024-react-vs-vue-vs-svelte-which-one-to-choose-4c0p>.
- [4] A. Vasylenko. “Vue vs react: A complete 2026 comparison for scalable web apps,” The Frontend Company, Accessed: Apr. 15, 2026. [Online]. Available: <https://www.thefrontendcompany.com/posts/vue-vs-react>.
- [5] A. Perini. “State management in front-end web development: State 101,” DEV Community, Accessed: May 8, 2026. [Online]. Available: <https://dev.to/abbeyperini/state-management-in-front-end-web-development-state-101-48g3>.
- [6] A. Tomar. “React prop drilling: Should you use it?” Mindroast, Accessed: Apr. 15, 2026. [Online]. Available: <https://www.mindroastblog.com/post/react-prop-drilling-should-you-use-it>.
- [7] E. Putans. “Single source of truth and applying it software development,” Medium, Accessed: May 8, 2026. [Online]. Available: <https://edunceputans.medium.com/single-source-of-truth-and-problems-with-implication-in-an-organisation-588883492133>.

- [8] Fslunsky. "State vs. props vs. context vs. redux: Why is managing data in react so confusing?" Medium, Accessed: Apr. 15, 2026. [Online]. Available: <https://medium.com/@fslunsky/state-vs-props-vs-context-vs-redux-why-is-managing-data-in-react-so-confusing-7481cf1209b4>.
- [9] N. Kramer. "Bootstrap vs tailwindcss," daily.dev, Accessed: Apr. 19, 2026. [Online]. Available: <https://daily.dev/blog/bootstrap-vs-tailwindcss>.
- [10] B. Koya. "Real-time communication mastery: Long polling vs websockets in .net-part 1:your complete developer's journey," Medium, Accessed: Apr. 15, 2026. [Online]. Available: <https://medium.com/@bhargavkoya56/real-time-communication-mastery-long-polling-vs-websockets-in-net-part-075619866e7a>.
- [11] J. Mataić. "Asp.net core signalr vs. websockets: Comparing uncomparable," Notch, Accessed: Apr. 15, 2026. [Online]. Available: <https://wearenotch.com/blog/signalr-websockets/>.
- [12] PubNub. "What is signalr framework?" PubNub, Accessed: May 8, 2026. [Online]. Available: <https://www.pubnub.com/guides/signalr/>.
- [13] A. Solanki. "Backend battle: .net vs python in 2025—which one wins?" iBrand-Studio, Accessed: May 8, 2026. [Online]. Available: <https://ibrandstudio.com/articles/backend-battle-net-vs-python>.
- [14] IBM. "Containers versus virtual machines (vms): What's the difference?" Accessed: Apr. 19, 2026. [Online]. Available: <https://www.ibm.com/think/topics/containers-vs-vms>.
- [15] M. Academy. "Docker vs podman: An in-depth comparison (2026)," DEV Community, Accessed: Apr. 25, 2026. [Online]. Available: https://dev.to/mechcloud_academy/docker-vs-podman-an-in-depth-comparison-2025-2eia.
- [16] J. Tao. "Characteristics of time series data," TDengine, Accessed: Apr. 16, 2026. [Online]. Available: <https://tdengine.com/characteristics-of-time-series-data/>.
- [17] J. Wachtel. "Don't let time series data break your relational database," InfluxData, Accessed: May 8, 2026. [Online]. Available: <https://www.influxdata.com/blog/dont-let-time-series-data-break-your-relational-database/>.

- [18] G. Gattani. “Why a time-series db (like influxdb) beats a “normal” db for time-stamped data,” Medium, Accessed: Apr. 16, 2026. [Online]. Available: <https://medium.com/@gayatrigattani2001/why-a-time-series-db-like-influxdb-beats-a-normal-db-for-time-stamped-data-e50da8522d34>.

Appendix A

Instructions to Compile and Run the System

This appendix describes how to set up, configure, and run the ground control system. The purpose is to make it easier for future students or members of UiS Aerospace to deploy, test, and continue development of the system.

A.1 System Requirements

The only requirement to run the system using Docker is:

- Docker and Docker compose

For running the backend and frontend separately during development, the following is also required:

- .NET 9 SDK
- Node.js and npm

A.2 Project Structure

The repository contains two Docker compose files, one for each deployment mode:

- `compose.operator.yaml` — starts the operator frontend, operator backend, and a local InfluxDB instance
- `compose.sponsor.yaml` — starts the sponsor frontend, sponsor backend, and a cloud InfluxDB instance

Both files build from the same source code, but inject different environment variables to configure the mode, database connections, and network settings.

A.3 Configuration

All configuration is handled through environment variables in the Docker Compose files. The key variables for the operator version are shown below:

```
AppMode=Operator
CommandServerIP=host.docker.internal
CommandServerPort=8090
CommandKey=123
InfluxDBLocal__Url=http://phoenix_influx_local:8086
InfluxDBLocal__Token=my-super-secret-local-token
InfluxDBLocal__Org=UiSAerospace
InfluxDBLocal__Bucket=PhoenixLocal
InfluxDBCloud__Url=http://192.168.0.184:8086
InfluxDBCloud__Token=my-super-secret-cloud-token
InfluxDBCloud__Org=UiSAerospace
InfluxDBCloud__Bucket=PhoenixCloud
```

Before deploying, update `InfluxDBCloud__Url` to the actual IP address of the cloud server, and replace the tokens with secure values.

A.4 Running the Operator Version

1. Clone the repository.
2. Navigate to the project root folder.
3. Update the environment variables in `compose.operator.yaml` if needed.
4. Start the system:

```
docker compose -f compose.operator.yaml up --build
```

The operator frontend will be available at `http://localhost:8080` and the backend API at `http://localhost:8087`.

A.5 Running the Sponsor Version

1. Navigate to the project root folder.
2. Update the environment variables in `compose.sponsor.yaml` if needed.
3. Start the system:

```
docker compose -f compose.sponsor.yaml up --build
```

The sponsor frontend will be available at `http://localhost:8080` and the backend API at `http://localhost:8087`.

A.6 Running the Backend and Frontend Separately

During development, the backend and frontend can be started without Docker.

A.6.1 Backend

```
cd Backend/API  
dotnet run
```

A.6.2 Frontend

```
cd Frontend
npm install
npm run dev
```

A.7 Replay Functionality

To use replay mode, the system must first be running with telemetry data stored in the database from a previous session. Replay is then started through the operator frontend by selecting a recorded session from the replay controller in the navigation bar. The backend will switch from `TcpSource` to `InfluxSource` and stream the stored data through the same pipeline as live data.

A.8 Troubleshooting

Common issues and their likely causes:

- **Port conflict** — ports 8080, 8086, or 8087 are already in use. Stop the conflicting service or change the port mappings in the Compose file.
- **InfluxDB connection failure** — verify that the `Url`, `Token`, `Org`, and `Bucket` values match the running InfluxDB instance.
- **No telemetry data received** — verify that the TCP source is sending data to port 8088 on the host machine.
- **Commands not working** — verify that the `CommandKey` environment variable matches the key entered in the operator frontend.

If issues persist, inspect the container logs with:

```
docker compose -f compose.operator.yaml logs
```

Appendix B

Adding New Charts and Dashboard Elements

A goal of this project was not only to create a ground control system for the current rocket, but also to make the software reusable for future systems and missions. Future UiS Aerospace projects may require new dashboards, new telemetry values, or use of the same software structure in other technical systems. This appendix is therefore written to encourage reuse of the program by explaining how existing frontend components can be combined and adapted without rewriting the underlying telemetry logic.

B.1 Overview

The frontend is built using reusable components. Instead of creating each chart or value display from scratch, a new dashboard page can be created by combining existing components. Examples of reusable components include:

- `LineChart` — displays telemetry values over time
- `BarChart` — displays telemetry values as a bar chart
- `FuelTankChart` — displays tank level information for fuel or oxidizer tanks

- Card — groups related dashboard elements visually
- Button — used for user interaction
- TextData — displays the latest value of a telemetry signal
- TinyChart or TinyBarChart — displays a compact chart

Most telemetry components receive a `sensorId` property. This value tells the component which telemetry signal to read from the shared Redux store. For example, using `sensorId="Temperature"` will make the component display data connected to the temperature signal.

B.2 Creating a New Dashboard Page

A new dashboard page can be created by adding a new React component inside the frontend page or view folder. The page should import the reusable components that are needed and then arrange them using React JSX. Styling and layout are handled through Tailwind CSS utility classes written in the `className` attribute.

The following simplified example shows how a dashboard page can be built by combining a header, cards, a line chart and text displays:

Listing B.1: Simple dashboard page using reusable frontend components

```
1 import Header from "../components/Header.tsx";
2 import Card from "../components/Card.tsx";
3 import TextData from "../components/TextData.tsx";
4 import LineGraphChart from "../components/LineGraphChart.tsx";
5
6
7 export default function Propulsion() {
8   return (
9     <div className="bg-[#121212] min-h-screen flex flex-col
10       ">
11       <Header pageName="Propulsion" />
12       <Card className="m-4">
```

```

13         <LineGraphChart sensorId="Temperature" />
14     </Card>
15
16     <Card className="m-4">
17         <TextData sensorId="Temperature" />
18     </Card>
19 </div>
20 );
21 }

```

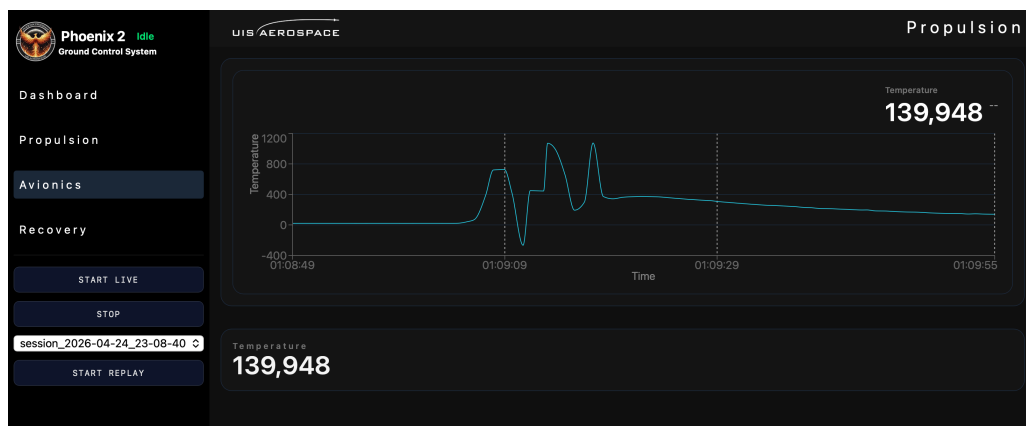


Figure B.1: Result of the code

B.3 Using Existing Components

To add a new chart or data display, first import the component at the top of the file. For example:

Listing B.2: Importing reusable frontend components

```

1 import TextData from "../components/TextData.tsx";
2 import Card from "../components/Card.tsx";
3 import BarChart from "../components/BarChart.tsx";
4 import FuelTankChart from "../components/FuelTankChart.tsx";

```

The component can then be placed inside the returned JSX. If the component displays telemetry data, it should usually be given a `sensorId`:

Listing B.3: Using reusable telemetry components with sensor identifiers

```
1 <TextData sensorId="Temperature" />
2 <BarChart sensorId="Temperature" />
3 <FuelTankChart sensorId="fuellevel" fuelColor="#f59e0b" />
```

The `sensorId` must match the name used by the backend and Redux store for that telemetry value. If the name does not match, the component will not receive the correct data.

B.4 Grouping Components with Cards

The Card component is used to group related information and give the dashboard a consistent visual structure. For example, several temperature values can be placed inside one card:

Listing B.4: Grouping telemetry values inside a card

```
1 <Card className="m-4">
2   <p className="uppercase tracking-[0.25em] text-white font-
3     semibold mb-4">
4     Temperature Combustion Chamber
5   </p>
6   <div className="flex justify-between">
7     <TextData sensorId="Temperature" />
8     <TextData sensorId="Temperature" />
9   </div>
10 </Card>
```

The `className` property can be used to add Tailwind CSS classes for spacing, layout, and alignment.

B.5 Adding a New Telemetry Value

To display a new telemetry value in the dashboard, follow these steps:

1. Make sure the backend receives and forwards the new telemetry signal.
2. Make sure the telemetry value is stored in the Redux store.
3. Find the correct telemetry identifier for the signal.
4. Add an existing component to the dashboard.
5. Set the component's `sensorId` to the telemetry identifier.

For example, if a new telemetry signal called `Pressure` is available, it can be displayed using:

Listing B.5: Displaying a new pressure telemetry value

```
1 <TextData sensorId="Pressure" />
2 <BarChart sensorId="Pressure" />
```

If the value represents a tank level, it can be displayed with the `FuelTankChart` component:

Listing B.6: Displaying tank level telemetry with `FuelTankChart`

```
1 <FuelTankChart sensorId="Pressure" fuelColor="#f59e0b" />
```

If a line chart component is available, the same signal can also be shown as a graph over time:

Listing B.7: Displaying telemetry as a line chart

```
1 <LineGraphChart sensorId="Pressure" />
```

B.6 Recommended Development Process

When adding a new chart or dashboard element, it is recommended to start with a simple `TextData` component to verify that the telemetry value is available. After

confirming that the correct value is displayed, more advanced components such as `LineChart`, `BarChart`, or `FuelTankChart` can be added.

A recommended workflow is:

1. Add `TextData` with the desired `sensorId`.
2. Run the frontend and verify that the value appears.
3. Replace or combine it with a chart component.
4. Place the component inside a `Card`.
5. Adjust layout and spacing using Tailwind CSS.
6. Test the dashboard with live or replayed telemetry data.

B.7 Troubleshooting

Common issues when adding new charts include:

- **No data displayed** — check that the `sensorId` matches the telemetry identifier exactly.
- **Chart appears empty** — verify that the telemetry signal contains data in the format expected by the chart component.
- **Fuel tank does not update** — check that the tank component receives the correct `sensorId` and that the value is within the expected range.
- **Component not visible** — check that it is placed inside the returned JSX and that the page is included in the routing system.
- **Layout looks wrong** — adjust the Tailwind CSS classes passed through `className`.
- **Import error** — verify that the component path is correct.

By reusing the existing components, future developers can add new dashboard elements without changing the underlying telemetry logic. This keeps the frontend modular and makes it easier to adapt the system to new sensors, subsystems, or mission requirements.

Appendix C

Excel Overview Sheet

GUI-dataoversikt, *Phoenix 2*

Gruppe/View	Navn	Beskrivelse	Ønsket fremstillingsmetode	Enhet	Data-format (inn til GUI fra telemetri)
Propulsion	Fu_level				TBD
	Ox_level	Hvor mye drivstoff har vi i tanken(e)?	Figur av en tank, med nivå som stiger, evt. kun en label med %	%	TBD
	P_Fu_Pump_In				TBD
	P_Fu_Pump_Out	Hva er trykket inn/ut i turbopumpen(e)?	Label	bar	TBD
	P_Ox_Pump_In				TBD
	P_Ox_Pump_Out				TBD
	P_Fu_Injector		Label		TBD
	P_Ox_Injector	Hva er trykket før injektoren(e)?	Label		TBD
	T_Chamb	Hva er temperatur inne i forbrenningskammer?	Label som endrer farger - (grønt, oransje, rødt) basert på threshold verdier.		TBD
	T_Wall_1	Temperatur i veggen på forbrenningskammeret.		°C	TBD
Avionikk Hardware	T_Wall_2				TBD
	T_Ox_Injector				TBD
	T_Fu_Injector	Temperatur før injektorer.			TBD
	Spennig_PSU	ADC for batspenning. 18-24V range			TBD
	T_Nosecone	Temperature in nosecone		°C	TBD
	Aks_g	Akselerasjon målt i G. 0-20 range ish			TBD
	Heading	Fra INS			TBD
					TBD
					TBD
					TBD
Avionikk Software	longitude_deg	rakettsens estimerte geografiske kordinater	kart med path? labels?	deg	double
	longitude_deg			deg	double
	altitude_m	Rakettsens estimerte høyde over havet		m	double
	yaw	Yaw angle relative to true north.	3d modell som viser orientering av rakett	deg	float32
	pitch	Pitch angle relative to horizon		deg	float32
	roll	Roll angle relative to horizon		deg	float32
	rocket_state_enum	enum for rocket state	Label	state	uint32_t

Ellers er vi veldig enig med software



University
of Stavanger

4036 Stavanger

Tel: +47 51 83 10 00

E-mail: post@uis.no

www.uis.no

Cover Photo: Hein Meling

© **2026 Lukas Fernandez Berntsen and Markus Solgård**